AGGIE CODING CLUB · MEETING 2

CUDA Vision

Beyond std:: — fancy iterators and 2D data on the GPU



RECAP · MEETING 1 IN PLAIN ENGLISH

Everything from last time, from zero

1. Host and device: two processors
2. C++: vectors and <templates>
3. C++: lambdas (mini functions)
4. C++: iterators and algorithms
5. The two GPU switches
6. Exercise 3, line by line

Recap: CPU vs GPU

In CUDA they get special names: the host and the device

HOST = the CPU

The main processor. Your program always starts here, in main().

A few very strong cores. Good at one complicated thing at a time.

Has its own memory (RAM).

⇄

DEVICE = the GPU

A helper processor. It only does the work the CPU hands it.

Thousands of small cores. Good at doing the same simple thing to lots of data at once.

Has its own separate memory.

How a CUDA program runs

The CPU runs your code line by line. When it reaches GPU work, it sends that job to the GPU, the GPU's threads each do one small piece, and then control comes back to the CPU.

Thread = one worker doing one small piece of the job. A GPU runs thousands of them at the same time.

1: a vector is a list of values

Reading this line one piece at a time

```
std::vector<float> temp{42, 24, 50};
```

std::
std is the C++ standard library.
:: is the scope resolution operator

vector
A list/array that can resize and has additional functionality.

<float>
This field is for the datatype of the values stored. Float is a decimal number.

temp
Name of object.

{42, 24, 50}
Initial values.

```
temp[0]       // 42  (starts at 0!)
temp[2]       // 50
temp.size()   // 3
```

GPU version: thrust::universal_vector<float>
Same idea, from the Thrust library. Its memory can be read by both the CPU and the GPU.

2: lambda functions

A lambda is a small function with no name, written right where you need it

```
auto op = [=](float t) { return t + k * (room - t); };
```

auto
Resolves the type at compile time.

op
Our variable name. Short for operation. We call it like op(42).

[=]
The capture. Copies outside variables (k, room) into the lambda so it can use them.

(float t)
The input. Like a normal function, it takes one float and calls it t.

{ return ... }
The body. Moves t halfway (k = 0.5) toward room temperature.

Try it with k = 0.5, room = 20:

```
op(42)   // 42 + 0.5 * (20 - 42) = 31
op(24)   // 24 + 0.5 * (20 - 24) = 22
```

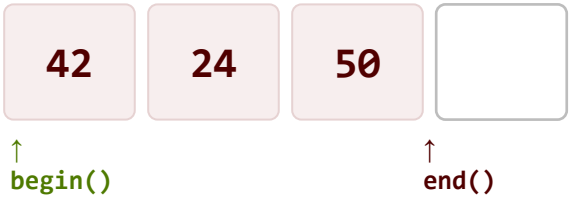
GPU version

`[=] __host__ __device__ (float t)`

Same lambda, with a label saying "make a GPU copy too."

3: algorithms work on ranges

begin() and end() mark where the data starts and stops



end() points one step past the last value, so the range is "start here, stop before there."

One line with an algorithm…

```
std::transform(temp.begin(), temp.end(),
               temp.begin(), op);
```

…does the same thing as this loop:

```
for (int i = 0; i < temp.size(); i++)
    temp[i] = op(temp[i]);
```

Arguments 1–2
The input range: from begin() to end().

Argument 3
Where to write results. Same place = update in place.

Argument 4
What to do to each value: our lambda op.

Why bother? A loop says HOW, one step at a time. An algorithm says WHAT, so the library is free to do it in parallel.

The two GPU switches

Every Thrust call needs both of these to agree

Switch 1: execution policy

`thrust::host` → run on the CPU

`thrust::device` → run on the GPU

The first argument to a Thrust algorithm. Picks WHERE it runs, decided while the program runs.

Switch 2: function specifier

`__host__` → build a CPU version

`__device__` → build a GPU version

A label on the lambda. Picks WHAT gets built, decided when you compile. Use both to get both.

	`__host__` only	`__device__` only	`__host__ __device__`
`thrust::host`	works (CPU)	compile error	works (CPU)
`thrust::device`	compile error	works (GPU)	works (GPU)

Kitchen analogy: specifier = recipe a kitchen knows; policy = which kitchen gets the order.

Putting it together: Exercise 3, line by line

A cooling simulation: every temperature moves halfway to room temperature

```
1  thrust::universal_vector<float> temp{42, 24, 50};
2  float k = 0.5f, room = 20;
3
4  auto op = [=] __host__ __device__ (float t) {
5    return t + k * (room - t);
6  };
7
8  thrust::transform(thrust::device,
9      temp.begin(), temp.end(), temp.begin(), op);
10 // temp is now {31, 22, 35}
```

1 Make a list of 3 temperatures that both CPU and GPU can read.

2 Two plain numbers. The lambda captures them with [=].

4–6 The rule for one value. __host__ __device__ = build a GPU copy.

8–9 Run op on every value, on the GPU, writing back in place.

To build and run it (in a Colab cell, start each line with !):

```
nvcc --extended-lambda -o ex3 ex3.cu      # 1. build
./ex3                                     # 2. run
```

`nvcc` = NVIDIA's compiler (like g++). **`--extended-lambda`** = allow __device__ lambdas. **`-o ex3`** = name of the program it builds.

Cheat sheet: words we use every meeting

Screenshot this one

`host / device`	CPU / GPU
`thread`	One worker doing one small piece of the job
`nvcc`	NVIDIA's C++ compiler, turns .cu files into programs
`namespace (std::)`	A "last name" saying which library a name is from
`template (<float>)`	Fills in a type, like what a vector holds
`lambda`	A small unnamed function you write inline

`capture ([=])`	Copies outside variables into a lambda
`iterator`	A bookmark into a list: begin(), end()
`algorithm`	A library function that works on a range: transform, sort…
`execution policy`	thrust::host or thrust::device: WHERE it runs
`specifier`	__host__ / __device__: WHAT gets compiled
`universal_vector`	A vector both the CPU and GPU can read

PART 3 · LEARNING

When the algorithm you need doesn't exist

1. Warm-up: median on the GPU
2. Memory bound: count the trips
3. Iterators: pointers with superpowers
4. Thrust fancy iterators
5. Vocabulary types: tabulate, pair, mdspan
6. 3 hands-on exercises

Exercise 4 (warm-up): median on the GPU

3 minutes · this median sorts on the CPU. Make the sort run on the GPU.

```
float median(thrust::universal_vector<float> vec)
{
    // TODO: make the code below execute on the GPU
    std::sort(vec.begin(), vec.end());
    return vec[vec.size() / 2];
}
```

Context: main() runs the cooling simulation for 3 steps with `thrust::transform` and prints the median temperature after each step.

Expected output

```
step  median
0     31.00
1     25.50
2     22.75
```

Hint: remember the two words you added last time.

File: `02-extending/ex4_median_problem.cu`

Exercise 4: solution

Before: CPU

```
std::sort(vec.begin(), vec.end());
```

After: GPU

```
thrust::sort(thrust::device, vec.begin(), vec.end());
```

std:: → thrust::

Thrust has GPU versions of the standard algorithms: sort, reduce, transform, copy, count, find…

+ thrust::device

Without the policy, Thrust picks based on the iterators. Being explicit makes the code easy to read.

Porting a standard algorithm is often just this. But what if the algorithm you need isn't in the library?

New question: how big was the change?

Each step, find the biggest change between the old and new temperatures

before	42	24	50
after	31	22	35
|change|	11	2	**15**

`max = 15`

There is no thrust::max_abs_difference

But we can build it from two algorithms we already know:

1. `transform` each pair → |a − b|
2. `reduce` with `thrust::maximum` → biggest one

Let's write that the obvious way first.

The obvious version: transform, then reduce

Correct… but it stores a whole extra vector just to throw it away

```
float naive_max_change(const universal_vector<float>& a,
                       const universal_vector<float>& b)
{
  universal_vector<float> diff(a.size());  // extra memory!

  thrust::transform(thrust::device,
      a.begin(), a.end(), b.begin(), diff.begin(),
      [] __host__ __device__(float x, float y) {
        return abs(x - y);
      });

  return thrust::reduce(thrust::device,
      diff.begin(), diff.end(),
      0.0f, thrust::maximum<float>{});
}
```

Two-input transform

Takes two sequences and applies the lambda to each pair (x, y).

reduce + maximum

Collapses a sequence to one value. Starts from 0.0f and keeps the bigger one.

The highlighted line is the problem: "unnecessarily materialized" differences, written to memory and read back once.

Count the trips to memory

Most GPU code is memory bound: moving data costs more than doing math on it

Naive version — read a | read b | write diff | read diff — **4n**

A plain for-loop — read a | read b — **2n**

```
float m = 0;
for (int i = 0; i < n; i++)
  m = max(m, abs(a[i] - b[i]));
```

Memory bound

Speed is limited by how fast memory can feed the cores, so half the trips ≈ twice as fast.

Goal: get the for-loop's 2n trips while still using library algorithms.

Iterators: a pointer you get to program

An iterator is anything you can index with [ ] or advance with ++

```
std::array<int, 3> a{0, 1, 2};
int* pointer = a.data();

pointer[0];   // 0  (reads memory)
pointer[1];   // 1  (reads memory)
```

A pointer reads values stored in memory

```
struct counting_iterator {
  int operator[](int i) { return i; }
};

counting_iterator it;
it[0];   // 0  (no memory at all!)
it[1];   // 1
```

A counting iterator computes them

C++ lets us redefine `operator[]`. So "reading" from an iterator can run any code we want.

An infinite sequence 0, 1, 2, 3, … without allocating a single byte.

Two more building blocks: transform and zip

Hand-written versions, so you can see there is no magic

transform: apply a function on read

```
struct transform_iterator {
  int* a;
  int operator[](int i) {
    return a[i] * 2;
  }
};   // a = {0,1,2}: it[1] → 2
```

zip: read two sequences as pairs

```
struct zip_iterator {
  int* a; int* b;
  std::tuple<int,int> operator[](int i) {
    return {a[i], b[i]};
  }
};   // it[0] → (a[0], b[0])
```



Nest them: a transform iterator wrapping a zip iterator gives the absolute difference of element i, **computed only when someone reads it**. Nothing is stored.

Thrust's fancy iterators

Library versions of what we just built. They work on the GPU and plug into any algorithm.

thrust::…	Looks like	Use it to…
`make_counting_iterator(0)`	`0, 1, 2, 3, …`	Indices without storing them
`make_transform_iterator(it, f)`	`f(it[0]), f(it[1]), …`	Apply a function on read
`make_zip_iterator(a, b)`	`(a[0],b[0]), (a[1],b[1]), …`	Walk several sequences at once
`make_discard_iterator()`	`writes go nowhere`	Ignore outputs you don't need
`make_transform_output_iterator`	`f(value) on write`	Change values as they are stored

All live in <thrust/iterator/…>. Full reference: nvidia.github.io/cccl/thrust

Exercise 5: variance without extra memory

5 minutes · build the squared differences with a transform iterator

$$\text{variance} = \Sigma\ (x_i - \text{mean})^2\ /\ N$$

```
float variance(const universal_vector<float>& x, float mean)
{
  // TODO: dereferencing squared_differences should give
  //       (xi - mean) * (xi - mean)
  auto squared_differences = ...;

  return thrust::reduce(thrust::device, squared_differences,
                        squared_differences + x.size()) / x.size();
}
```

Expected output

```
step  variance
0     29.56
1     7.39
2     1.85
```

Hint

Use **`thrust::make_transform_iterator(x.begin(), ...)`**. The lambda needs **`mean`**: capture it with **`[mean]`**.

File: `02-extending/ex5_variance_problem.cu`

Exercise 5: solution

```
auto squared_differences = thrust::make_transform_iterator(
    x.begin(),
    [mean] __host__ __device__(float value) {
      return (value - mean) * (value - mean);
    });
```

x.begin()

The sequence being transformed. Each read gives one temperature.

[mean]

Copies mean into the lambda, so the GPU has its own copy to use.

reduce sums it

Each squared difference is computed on read and added. Never stored.

Pattern: "compute per element, then combine" = transform iterator + reduce. One pass over memory.

2D data lives in 1D memory

Images and heat maps are grids, but GPU memory is one long row



grid: 3 rows × 4 cols



memory: row 0, then row 1, then row 2 ("row-major")

```
index = row * width + col     // (1, 2) → 1*4 + 2 = 6
row   = index / width;   col = index % width;
```

Every 2D GPU kernel does this math.

Our example: a 2D heat simulation. Each cell becomes the average of itself and its 4 neighbors.

One thread per cell: thrust::tabulate

tabulate(f) is exactly transform over a counting iterator

transform + counting_iterator

```
auto ids = thrust::make_counting_iterator(0);
thrust::transform(thrust::device,
    ids, ids + in.size(), out.begin(),
    [=] __host__ __device__(int id) {
      /* compute cell id */ });
```

thrust::tabulate

```
thrust::tabulate(thrust::device,
    out.begin(), out.end(),
    [=] __host__ __device__(int id) {
      /* compute cell id */ });
```

```
int column = id % width;   int row = id / width;
float d2tdx2 = in_ptr[row*width + column-1] - 2*in_ptr[row*width + column] + in_ptr[row*width + column+1];
float d2tdy2 = in_ptr[(row-1)*width + column] - 2*in_ptr[row*width + column] + in_ptr[(row+1)*width + column];
return in_ptr[row*width + column] + 0.2f * (d2tdx2 + d2tdy2);
```

The body (above) works, but count the **`row*width + column`**s. Easy to typo, hard to read. Let's clean it up.

Reuse code: std::pair… compile error?

Factor the row/col math into a helper and the GPU build complains

```
__host__ __device__
std::pair<int, int> row_col(int id, int width) {
  return std::make_pair(id / width, id % width);
}
```

```
error: calling a __host__ function("make_pair") from a __host__ __device__ function("row_col") is not allowed
```

```
__host__ __device__
cuda::std::pair<int, int> row_col(int id, int width) {
  return cuda::std::make_pair(id / width, id % width);
}
```

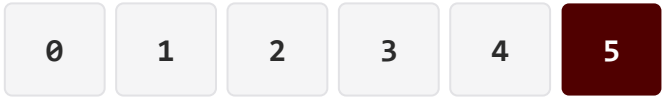
Fix: put `cuda::` in front. libcu++ gives you GPU-ready std:: types: pair, tuple, array, optional, span, mdspan…

mdspan: index a flat array like a grid

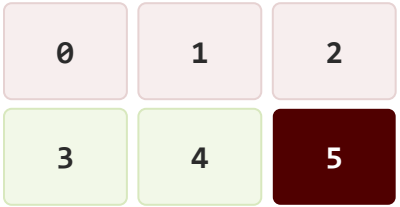
A lightweight view: a pointer + the shape. It owns no memory.

```
cuda::std::array<int, 6> sd{0, 1, 2, 3, 4, 5};
cuda::std::mdspan md(sd.data(), 2, 3);

md(0, 0);      // 0
md(1, 2);      // 5   (= sd[1*3 + 2])
md.extent(0);  // 2   height (rows)
md.extent(1);  // 3   width  (cols)
```



sd: flat memory



md: the same bytes, 2 × 3

Why it matters

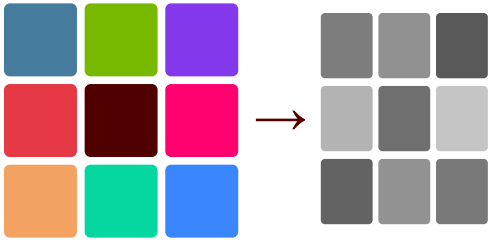
md(row, col) does the row*width + col math for you, carries its own width and height, and works in 3D too: md(row, col, channel) for RGB images.

Applying to the project: grayscale

A 3D mdspan: (row, col, channel). One tabulate, one thread per pixel.

```
// rgb: height * width * 3 floats, stored R,G,B,R,G,B,...
cuda::std::mdspan img(thrust::raw_pointer_cast(rgb.data()),
                      height, width, 3);

thrust::tabulate(thrust::device, gray.begin(), gray.end(),
  [img] __host__ __device__(int id) {
    int row = id / img.extent(1), col = id % img.extent(1);
    return 0.299f * img(row, col, 0)    // R
         + 0.587f * img(row, col, 1)    // G
         + 0.114f * img(row, col, 2);   // B
  });
```

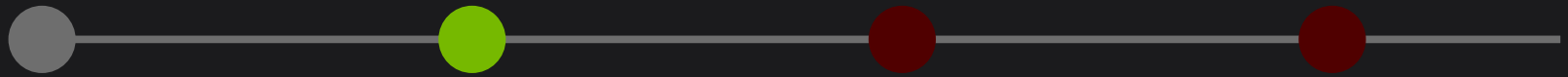


Same shape as everything today: **index → (row, col) → compute → store.**

Today's toolbox

`thrust::sort / reduce`	std:: algorithms on the GPU: swap the namespace, add the policy
`zip + transform iterators`	Build custom algorithms from library ones without temporary vectors
`Count memory trips`	GPU code is usually memory bound: fewer reads/writes = faster
`thrust::tabulate`	Call f(index) for every output element: one thread per cell or pixel
`cuda::std::pair, mdspan`	GPU-ready standard types. md(row, col) instead of pointer math

The road ahead



Thrust

Host vs device, std:: to thrust::

Iterators & mdspan

Fancy iterators, tabulate, 2D data (today)

Serial vs parallel

Why a for-loop inside a GPU lambda is slow. reduce_by_key

Grayscale

Simple image project

Before next meeting

Finish Exercises 4-5 if you didn't get to them. Try and make a pull request for github pushing answers for the exercises. Instructions in top of #resources in discord.